

Deep Learning — Final Exam Revision

ANN • CNN • RNN • LSTM • GRU • Gradient Descent • Momentum • AdaGrad • RMSProp

1. ANN — Artificial Neural Network

Basic neuron:

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

$$a = f(z)$$

Activations: Sigmoid → binary probability; Tanh → zero-centered; ReLU → hidden layers; Softmax → multi-class output.

Parameters: input × output + output (bias). Example: 10 → 5 gives $10 \times 5 + 5 = 55$.

Exam trick: Dense/Linear = every input connects to every neuron.

2. CNN

Convolution output size:

$$\text{Output} = \text{floor}((N + 2P - K) / S) + 1$$

N = input size, K = kernel, P = padding, S = stride.

CNN parameters:

$$(K_h \times K_w \times C_{in} + 1) \times C_{out}$$

Pooling usually has **0 trainable parameters**. Max Pool = maximum; Average Pool = average.

Memory: C-K-P-S-O → Channels, Kernel, Padding, Stride, Output.

3. RNN

Used for sequential data such as text, time series and speech.

$$\mathbf{h}_t = f(\mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{b})$$

Parameters: $DH + H^2 + H$, where D = input size and H = hidden size.

Important: RNN parameters are shared across time steps — do NOT multiply parameter count by sequence length.

Main problem: vanishing gradient; exploding gradient can also occur.

4. LSTM

LSTM handles long-term dependencies better than vanilla RNN.

$$\text{Forget: } f_t = \sigma(\mathbf{W}_f [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

$$\text{Input: } i_t = \sigma(\mathbf{W}_i [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

$$\text{Candidate: } \mathbf{C}_t = \tanh(\mathbf{W}_c [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$

$$\text{Cell: } \mathbf{C}_t = f_t \mathbf{C}_{t-1} + i_t \mathbf{C}_t$$

$$\text{Output: } o_t = \sigma(\mathbf{W}_o [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

$$\text{Hidden: } \mathbf{h}_t = o_t \tanh(\mathbf{C}_t)$$

Parameters: $4(DH + H^2 + H)$. Memory: Forget → Input → Candidate → Cell → Output → Hidden.

5. GRU

GRU is simpler than LSTM and has no separate cell state.

$$\text{Update: } z_t = \sigma(\mathbf{W}_z [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z)$$

$$\text{Reset: } r_t = \sigma(\mathbf{W}_r [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r)$$

$$\text{Candidate: } \mathbf{h}_t = \tanh(\mathbf{W}_h [r_t \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h)$$

Final: $h_t = (1-z_t) \cdot h_{t-1} + z_t \cdot h_t$

Parameters: $3(DH + H^2 + H)$. Memory: Update $\rightarrow$ Reset $\rightarrow$ Candidate.

6. Gradient Descent

Goal: minimize the loss.

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \eta(\partial L / \partial \mathbf{w})$$

η = learning rate. Positive gradient $\rightarrow$ weight decreases. Negative gradient $\rightarrow$ weight increases.

Batch GD = entire dataset **SGD** = one sample **Mini-batch** = small batch.

7. Momentum

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \mathbf{g}_t$$

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \eta \mathbf{v}_t$$

Idea: remembers previous direction, speeds consistent movement and reduces oscillation. Typical $\beta = 0.9$.

8. AdaGrad

$$\mathbf{G}_t = \mathbf{G}_{t-1} + \mathbf{g}_t^2$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{g}_t / (\sqrt{\mathbf{G}_t} + \epsilon)$$

Memory: AdaGrad = Accumulate squared gradients. Learning rate keeps decreasing and may become too small.

9. RMSProp

$$\mathbf{E}[\mathbf{g}^2]_t = \beta \mathbf{E}[\mathbf{g}^2]_{t-1} + (1-\beta) \mathbf{g}_t^2$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{g}_t / (\sqrt{\mathbf{E}[\mathbf{g}^2]_t} + \epsilon)$$

Memory: RMSProp = exponential moving average of recent squared gradients. Typical $\beta = 0.9$.

10. Optimization — One-Table Revision

Algorithm	Formula / Key Idea
Vanilla GD	$\mathbf{w} = \mathbf{w} - \eta \mathbf{g}$ — basic gradient
Momentum	$\mathbf{v} = \beta \mathbf{v} + \mathbf{g}$; $\mathbf{w} = \mathbf{w} - \eta \mathbf{v}$ — remembers past
AdaGrad	$\mathbf{G} = \mathbf{G} + \mathbf{g}^2$ — accumulates
RMSProp	$\mathbf{E} = \beta \mathbf{E} + (1-\beta) \mathbf{g}^2$ — moving average

■ 30-Second Memory Sheet

ANN: $y = f(\mathbf{W}\mathbf{x} + \mathbf{b})$; parameters = in×out+out

CNN: output = floor((N+2P-K)/S)+1; params = $(K_h K_w C_{in} + 1) C_{out}$

RNN: $DH + H^2 + H$

LSTM: $4(DH + H^2 + H)$ — has cell state

GRU: $3(DH + H^2 + H)$ — no separate cell state

GD: $\mathbf{w} = \mathbf{w} - \eta \mathbf{g}$

Momentum: $\mathbf{v} = \beta \mathbf{v} + \mathbf{g}$

AdaGrad: $\mathbf{G} = \mathbf{G} + \mathbf{g}^2$

RMSProp: $\mathbf{E} = \beta \mathbf{E} + (1-\beta) \mathbf{g}^2$

■ Final Exam Traps

1. Bias is included in parameter calculations.
2. RNN parameters are not multiplied by sequence length.

3. LSTM = 4× formula; GRU = 3× formula.
4. Pooling = 0 trainable parameters.
5. Positive gradient → weight decreases; negative gradient → weight increases.
6. AdaGrad = cumulative sum; RMSProp = exponential moving average.
7. Momentum uses previous velocity.
8. CNN output formula includes floor and +1.
9. Sigmoid → binary; Softmax → multiclass; ReLU → hidden layers.
10. LSTM has a cell state; GRU does not.
11. η = learning rate; β = momentum/decay coefficient.